



Rajshahi University of Engineering & Technology

Department of Computer Science & Engineering

Course No:3202

Course Name: Software Engineering Sessional

Experiment No:03

Experiment Name: Proxy Pattern and Flyweight Pattern

Submitted by: Name: Jayed Hasan Roni,Rafiul Islam Ovi,Md Abdus Salam Roll: 2203073,2203074,2203075 Section: B Series:22	Submitted to : Emrana Kabir Hashi Assistant Professor, Dept of Computer Science and Engineering, RUET
--	--

Date of Submission :19.08.2026

Lab Report: Proxy Pattern

1. Pattern Name

Proxy Pattern

2. Category

Structural Design Pattern

3. Intent

The Proxy Pattern provides a surrogate, placeholder, or gatekeeper for another object to control access to it. It allows you to perform operations either before or after the request reaches the original service object.

4. Problem Statement

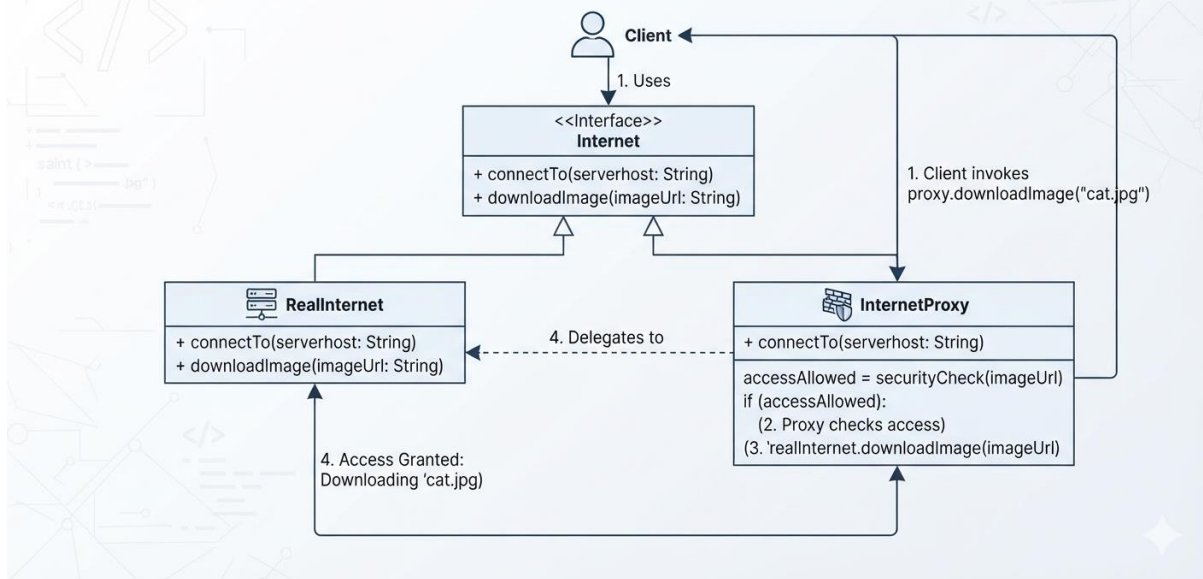
Imagine developing an **Internet Access Gateway** for an office network. Some websites (e.g., social media or malicious domains) must be blocked for security and productivity reasons. If every user connection directly hits the main internet server without filtering, the system violates security guidelines and lacks control.

5. Motivation

Directly altering the core internet service class to include blocklists introduces tightly coupled code and violates the **Single Responsibility Principle**. A normal direct implementation cannot dynamically restrict access, log requests, or lazy-load resource-intensive servers without altering core network logic.

6. Pattern Structure (Conceptual)

C++ Design Patterns: Proxy Pattern



7. Class Responsibilities

- **Internet (Subject Interface):** Defines a common blueprint interface for both the Real Object and the Proxy.
- **RealInternet (Real Subject):** Implements the actual core service logic (connecting to websites).
- **InternetProxy (Proxy):** Maintains a reference to the RealInternet object, contains a blocklist of restricted URLs, checks authorization, and controls access to the RealInternet.

8. Code Implementation (C++)

cpp

```
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>
```

```
using namespace std;
```

```
// 1. Subject Interface
```

```
class Internet {
```

```
public:
```

```
    virtual void connectTo(string serverHost) = 0;
```

```
    virtual ~Internet() {}
```

```
};
```

```

// 2. Real Subject (The expensive or protected service)
class RealInternet : public Internet {
public:
    void connectTo(string serverHost) override {
        cout << "Connecting successfully to network host: "
<< serverHost << "\n";
    }
};

// 3. Proxy Object (Acts as the gatekeeper)
class InternetProxy : public Internet {
private:
    RealInternet* realInternet;
    vector<string> blockedSites;

public:
    InternetProxy() {
        realInternet = new RealInternet();
        // Initialize the blocklist
        blockedSites.push_back("facebook.com");
        blockedSites.push_back("instagram.com");
        blockedSites.push_back("malicious-site.io");
    }

    ~InternetProxy() {
        delete realInternet;
    }

    void connectTo(string serverHost) override {
        // Convert to lowercase for uniform verification
        string host = serverHost;
        transform(host.begin(), host.end(),
host.begin(), ::tolower);

        // Check if the site is blocked
        if (find(blockedSites.begin(), blockedSites.end(),
host) != blockedSites.end()) {
            cout << "ACCESS DENIED: Connection to '" <<
serverHost << "' is restricted by company policy!\n";
        } else {
            // Forward request to the real service if
valid
            realInternet->connectTo(serverHost);

```

```

    }
}
};

```

9. Execution Flow / Demonstration

```

cpp
int main() {
    // Client interacts with the proxy seamlessly through
the Interface
    Internet* officeNetwork = new InternetProxy();

    cout << "--- Office Internet Browsing Session ---\n";

    // Test Case 1: Permitted Site
    officeNetwork->connectTo("google.com");

    // Test Case 2: Blocked Site
    officeNetwork->connectTo("facebook.com");

    // Test Case 3: Permitted Site
    officeNetwork->connectTo("github.com");

    delete officeNetwork;
    return 0;
}

```

10. Advantages

- **Security & Control (Protection Proxy):** Allows managing access permissions seamlessly without altering the core service client.
- **Open/Closed Principle:** New types of proxies (like logging or caching) can be introduced without modifying the real service or client code.
- **Lifecycle Management:** The proxy can manage the lifecycle of the real service object (instantiating it only when needed).

11. Limitations

- **Increased Latency:** Introducing an intermediary layer can add response overhead for requests.

- **Code Complexity:** Requires adding multiple new classes, which could overcomplicate simpler architectures.

12. Real-life Applications

- **Firewalls & Network Proxies:** Restricting unauthorized access to backend hosts.
- **Smart Pointers (C++):** Intermediary wrappers managing reference counting and memory safety for standard object pointers.
- **Hibernate/ORM Lazy Loading:** Postponing database object fetching until specific data attributes are requested.

13. Industry Examples

- **Nginx / Reverse Proxies:** Managing load balancing, encryption, and caching for high-traffic web applications.
- **Spring Framework (@Transactional):** Uses dynamic proxies to run transactions automatically around core business routines.
- **Credit Cards:** Acts as a financial proxy to abstract the actual handling of large cash amounts.

14. Sample Output

```
HP@DESKTOP-LT90DHK UCRT64 ~  
$ cd /c/cpp_local  
  
HP@DESKTOP-LT90DHK UCRT64 /c/cpp_local  
$ g++ ProxyPattern.cpp -o ProxyPattern.exe  
  
HP@DESKTOP-LT90DHK UCRT64 /c/cpp_local  
$ ./ProxyPattern.exe  
--- Office Internet Browsing Session ---  
Connecting successfully to network host: google.com  
ACCESS DENIED: Connection to 'facebook.com' is restricted by company policy!  
Connecting successfully to network host: github.com
```

15. Conclusion

The Proxy Pattern introduces a structured checkpoint system by inserting a placeholder object between the caller and the actual target. It helps developers enforce security boundaries, caching, and lazy loading natively, keeping systems modular and compliant with modern software engineering paradigms.

